Homework 11

Steven Jia

November 8, 2026

Question 1: Implementing K-means

(a) One iteration

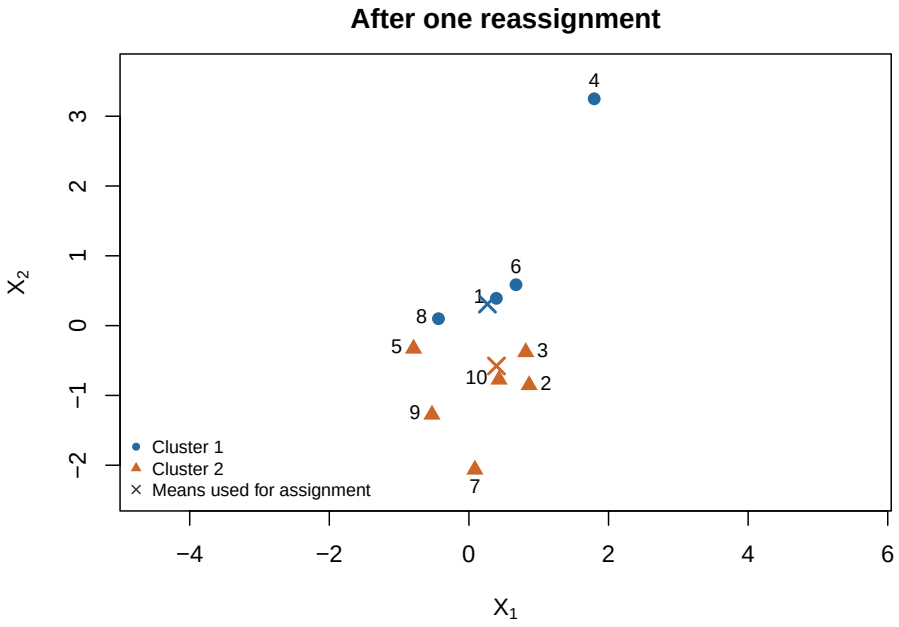
I used seed **43211** to generate 10 observations with two independent standard normal coordinates. With seed **43212**, I randomly assigned five points to each of two clusters. Both clusters were nonempty.

Point	X_1	X_2	Initial cluster	Updated cluster
1	0.393	0.391	2	1
2	0.863	-0.852	2	2
3	0.814	-0.380	2	2
4	1.795	3.249	1	1
5	-0.793	-0.330	1	2
6	0.674	0.586	1	1
7	0.086	-2.063	1	2
8	-0.437	0.099	1	1
9	-0.529	-1.275	2	2
10	0.431	-0.773	2	2

The means calculated from the **initial assignments** were:

Cluster	Mean of X1	Mean of X2
1	0.265	0.308
2	0.394	-0.578

I assigned each point to its closest mean using Euclidean distance. The updated assignments are shown above and in the plot. The crosses mark the initial means used for this assignment step.



(b) My K-means function

The function below works with any number of columns. It stops when the assignments stop changing or `max_iter` is reached. If a cluster becomes empty, it keeps its previous mean. The final means are recalculated before returning the result. Ties go to the lowest cluster number.

```r
# Each initial cluster has at least one observation.
random_assignment <- function(n, K) {
  sample(rep(seq_len(K), length.out = n))
}

my_kmeans <- function(x, K, initial = NULL, max_iter = 300L) {
  x <- as.matrix(x)
  n <- nrow(x)
  p <- ncol(x)
  stopifnot(is.numeric(x), n >= 1L, p >= 1L, all(is.finite(x)),
            length(K) == 1L, K == as.integer(K), K >= 1L, K <= n,
            length(max_iter) == 1L, max_iter >= 1L,
            max_iter == as.integer(max_iter))
  if (is.null(initial)) initial <- random_assignment(n, K)
  stopifnot(length(initial) == n, all(initial %in% seq_len(K)),
            all(tabulate(initial, nbins = K) > 0L))
  cluster <- as.integer(initial)
  centers <- matrix(0, K, p)

  update_means <- function(cluster, previous) {
    for (k in seq_len(K)) {
      members <- which(cluster == k)
      if (length(members) > 0L)
        previous[k, ] <- colMeans(x[members, , drop = FALSE])
    }
    previous  # Empty clusters retain their previous mean.
  }
  centers <- update_means(cluster, centers)
  objective <- function(cluster, centers) {
    sum((x - centers[cluster, , drop = FALSE])^2)
  }
  history <- objective(cluster, centers)
  converged <- FALSE

  for (iteration in seq_len(max_iter)) {
    distances <- matrix(0, n, K)
    for (k in seq_len(K))
      distances[, k] <- rowSums(sweep(x, 2, centers[k, ], "-")^2)
    # Squared Euclidean distance has the same nearest center.
    next_cluster <- max.col(-distances, ties.method = "first")
    converged <- identical(cluster, next_cluster)
    cluster <- next_cluster
    centers <- update_means(cluster, centers)
    history <- c(history, objective(cluster, centers))
    if (converged) break
  }
  centers <- update_means(cluster, centers)
  colnames(centers) <- colnames(x)
  rownames(centers) <- paste0("Cluster ", seq_len(K))
  list(cluster = cluster, centers = centers,
       wcss = objective(cluster, centers), iterations = iteration,
       converged = converged, history = history)
}
```

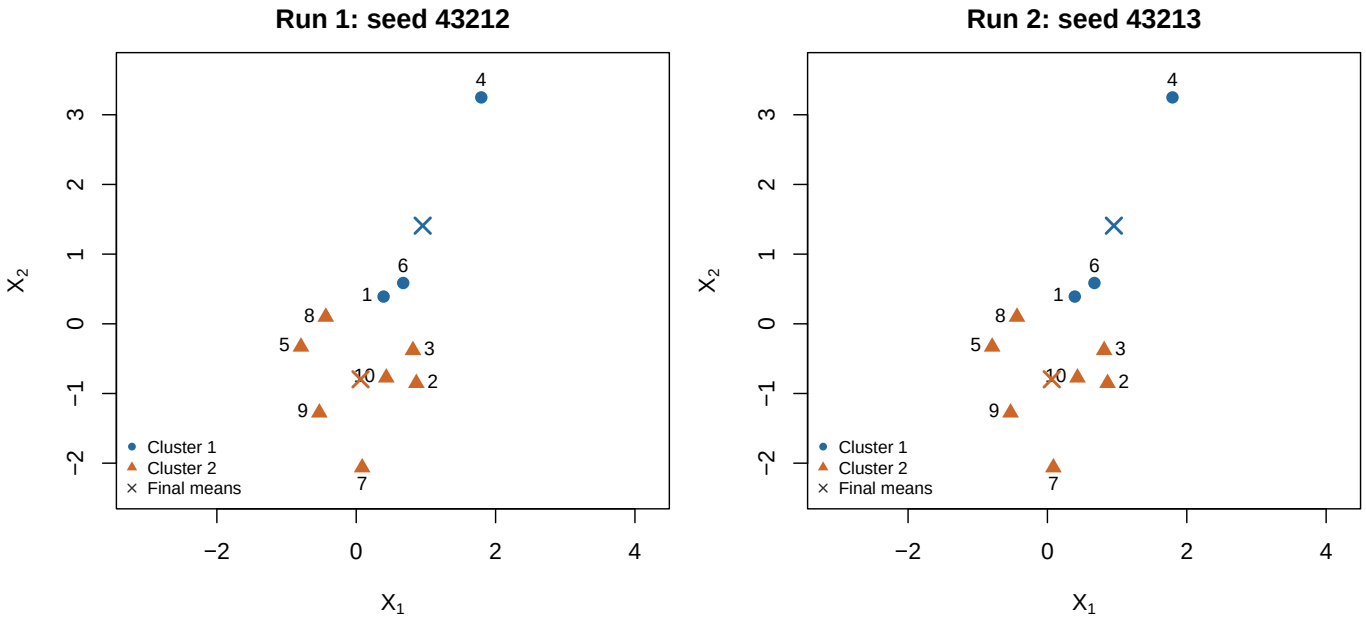
The returned `wcss` is $\sum_{k=1}^{K} \sum_{i:C(i)=k} \|x_i - m_k\|^2$.

(c) Two random initial assignments

I used the same 10 observations for both runs. Run 1 started with 2, 2, 2, 1, 1, 1, 1, 1, 2, 2 (seed **43212**). Run 2 started with 2, 1, 2, 1, 2, 1, 1, 1, 2, 2 (seed **43213**). Both used a maximum of 300 iterations.

Run	Iterations	WCSS
1	3	11.90389
2	3	11.90389

Run	Cluster	Mean of X1	Mean of X2
1	1	0.954	1.409
1	2	0.062	-0.796
2	1	0.954	1.409
2	2	0.062	-0.796



Both runs gave the **same grouping and the same WCSS**. In both runs, cluster 1 contained points 1, 4, 6, and cluster 2 contained points 2, 3, 5, 7, 8, 9, 10. The two starting assignments led to the same final solution for these data.

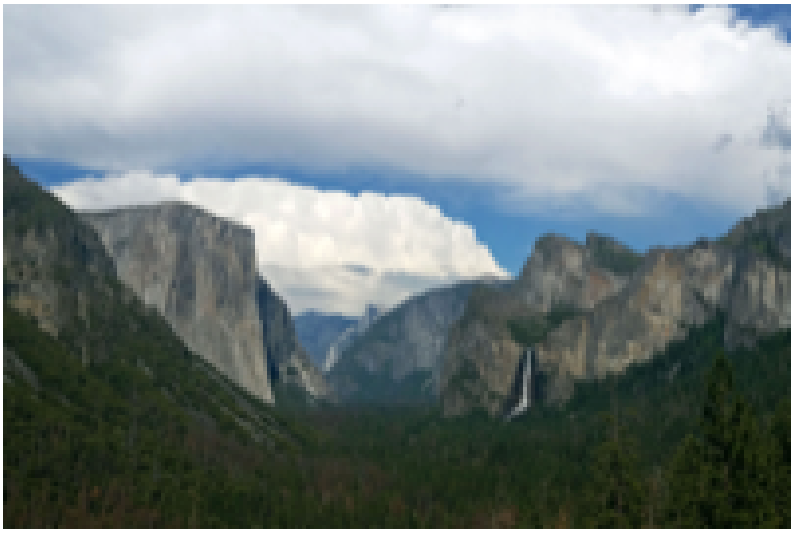
This does not mean different initial assignments always give the same answer. K-means can stop at different local solutions, so trying several starts can help find a smaller WCSS. Also, switching cluster numbers alone would not count as a different grouping.

Question 2: Image color compression

(a) Image and pixel matrix

I chose a landscape photo of Yosemite Valley from Tunnel View. The original file is **2,880 pixels wide and 1,920 pixels high**. I resized it while keeping its proportions to **200 pixels wide and 133 pixels high**, so both dimensions are at most 200. The image used for clustering is shown below. Source: U.S. National Park Service.

Original image (resized to 200 x 133)



The resized image has $200 \times 133 = 26{,}600$ pixels. I converted it into a **26,600 by 3 matrix**, with one row per pixel and columns red, green, and blue. I used RGB values on the **0–1 scale** for every fit. I did not standardize the color channels. Pixel rows follow the image from left to right and top to bottom.

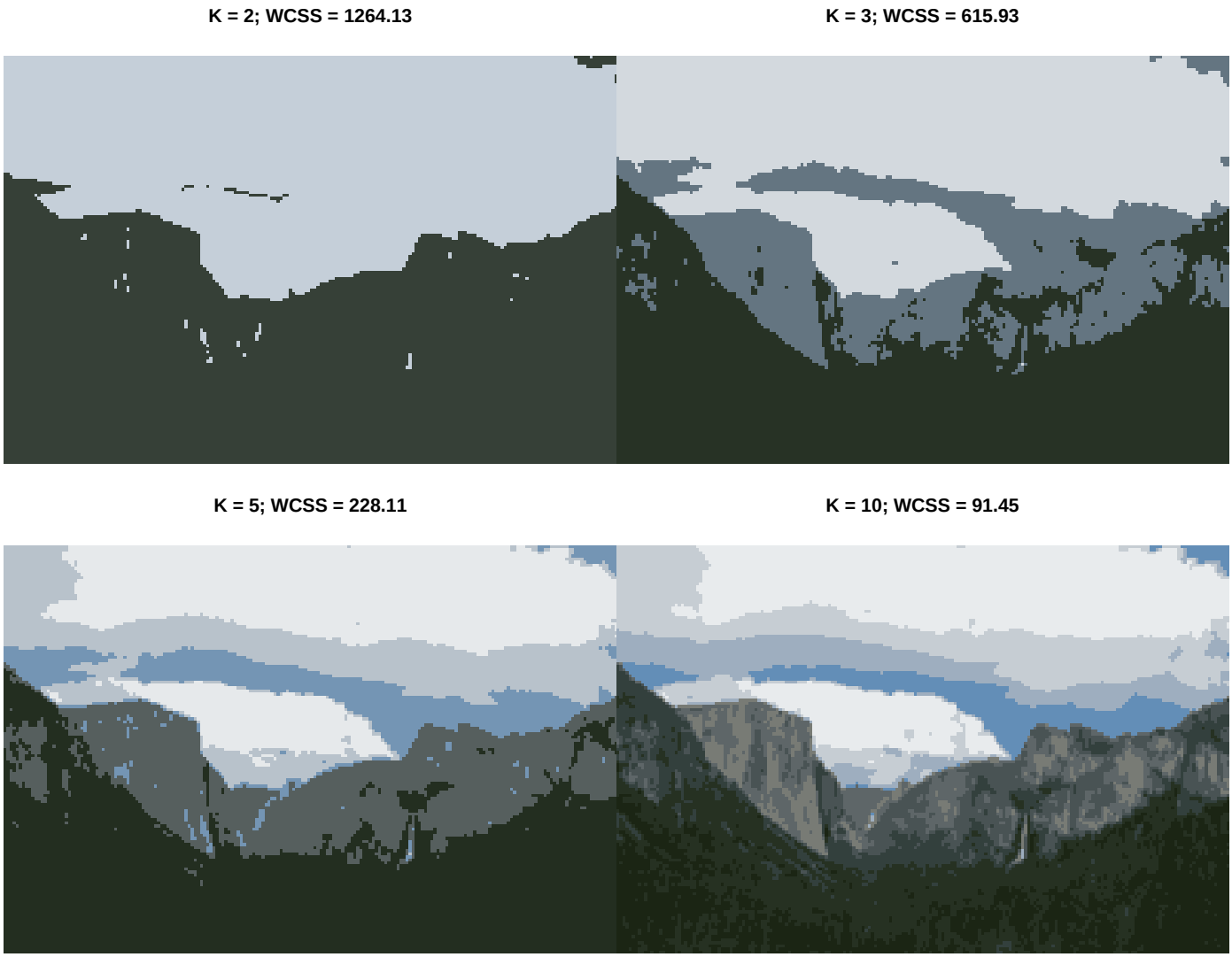
(b) K-means fits

I used my function from Question 1 for $K = 2, 3, 5, 10$. For each K, I tried five random nonempty initial assignments and kept the run with the lowest WCSS. The seed for start s was $43200+100K+s$, for $s = 1, \ldots, 5$. Every run converged within the maximum of 300 iterations.

K	Selected seed	WCSS	Iterations
2	43401	1264.126	5
3	43501	615.934	25
5	43701	228.106	64
10	44201	91.451	86

I reconstructed each image by replacing every pixel with its cluster's mean RGB color. The following page shows the results, followed by all cluster means.

(b) Reconstructed images and (c) my choice of K



I would choose $K = 10$ for this image. With two colors, most of the mountains and trees merge into one dark region. Three colors show more of the mountain outlines, but much of the detail is still missing. Five colors separate the blue sky, clouds, and darker landscape more clearly. Ten colors preserve more shading on the cliffs and more detail in the trees, so the image looks closer to the original photo.

The WCSS falls from **228.106** at $K = 5$ to **91.451** at $K = 10$, a decrease of about **59.9%**. Ten colors still form a small palette, and I prefer the extra detail here. This compares color approximation; WCSS does not measure the actual file size of the saved image.

(b) Cluster mean colors

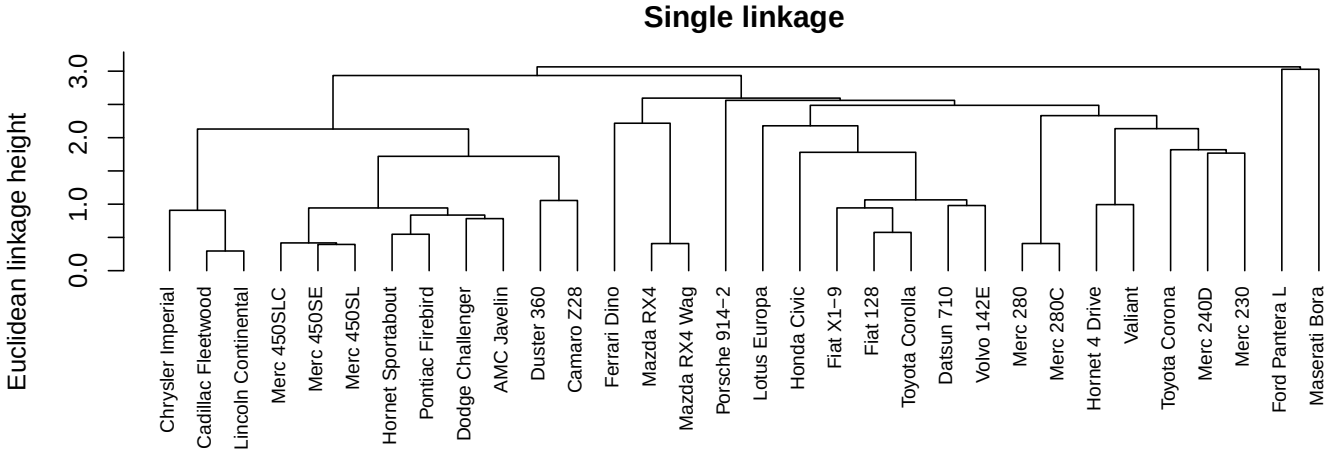
Each row is one cluster's mean color on the 0–1 RGB scale. Cluster numbers are arbitrary within each fit. Pixel counts are included to show how much of the image uses each color.

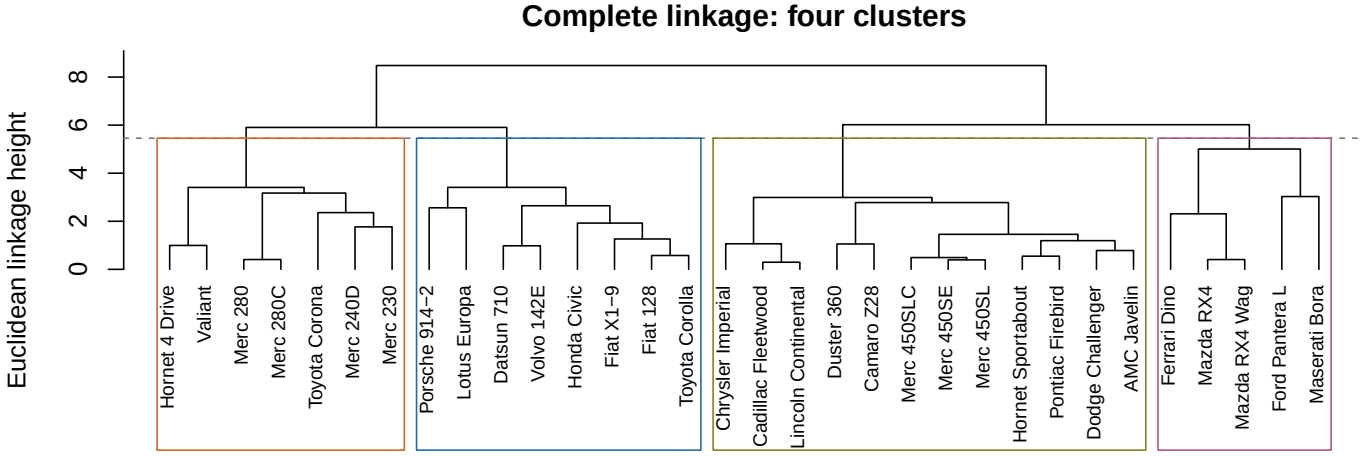
K	Cluster	Red	Green	Blue	Pixels
2	1	0.2101	0.2509	0.2171	14434
2	2	0.7730	0.8120	0.8495	12166
3	1	0.3931	0.4606	0.5053	5574
3	2	0.8268	0.8492	0.8715	10443
3	3	0.1523	0.1949	0.1465	10583
5	1	0.7226	0.7599	0.7974	3888
5	2	0.1387	0.1809	0.1259	9302
5	3	0.4534	0.5826	0.7045	2268
5	4	0.9011	0.9120	0.9226	6299
5	5	0.3372	0.3709	0.3694	4843
10	1	0.2870	0.3266	0.3279	1932
10	2	0.1498	0.1908	0.1341	4291
10	3	0.4705	0.4805	0.4582	975
10	4	0.3676	0.4010	0.4068	1690
10	5	0.7765	0.8028	0.8294	3348
10	6	0.2015	0.2520	0.2411	1824
10	7	0.3880	0.5571	0.7185	1338
10	8	0.9114	0.9211	0.9307	5507
10	9	0.1047	0.1457	0.0776	3635
10	10	0.6186	0.6838	0.7478	2060

Question 3: Hierarchical clustering of cars

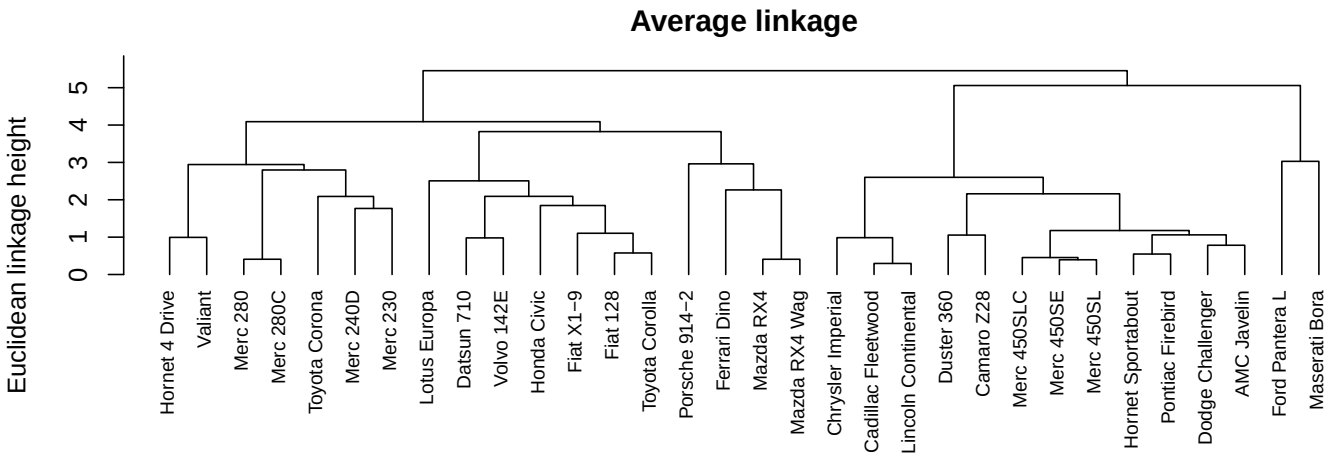
(a) Standardization and three linkage methods

I used all 11 numerical variables for the 32 cars. Car names were used only as labels. I standardized each variable as $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$, using its sample standard deviation, then calculated Euclidean distances. The three methods use the same standardized data and distance matrix.





(a) Average linkage and comparison



Single linkage uses the closest pair of cars between two groups. It shows a chaining pattern: groups can connect through a few nearby cars. For example, cutting this tree into three groups gives one group of 30 cars and two single-car groups, Ford Pantera L and Maserati Bora.

Complete linkage uses the farthest pair between groups. It gives more separated groups here and avoids the same large chain. Average linkage uses the average distance between all cross-group pairs. At three groups, average linkage puts Ford Pantera L and Maserati Bora together, while the other groups contain 18 and 12 cars. The exact grouping therefore depends on how distance between groups is defined.

(b) My choice: complete linkage with four clusters

I chose **complete linkage and four clusters**. In this tree, the merge from five groups to four happens at height **5.007**, while the next merge, from four groups to three, is at **5.905**. A cut between these heights preserves four groups with understandable differences. This is a reasonable descriptive choice, not proof that four is the only correct number. The dashed line in the complete-linkage plot marks this cut.

(b) Cluster memberships

Cluster 1 (5 cars): Mazda RX4; Mazda RX4 Wag; Ford Pantera L; Ferrari Dino; Maserati Bora.

Cluster 2 (8 cars): Datsun 710; Fiat 128; Honda Civic; Toyota Corolla; Fiat X1-9; Porsche 914-2; Lotus Europa; Volvo 142E.

Cluster 3 (7 cars): Hornet 4 Drive; Valiant; Merc 240D; Merc 230; Merc 280; Merc 280C; Toyota Corona.

Cluster 4 (12 cars): Hornet Sportabout; Duster 360; Merc 450SE; Merc 450SL; Merc 450SLC; Cadillac Fleetwood; Lincoln Continental; Chrysler Imperial; Dodge Challenger; AMC Javelin; Camaro Z28; Pontiac Firebird.

Cluster means on the original scales

The tables split the 11 variables into two panels for readability. All values are calculated from the original data, before standardization.

Cluster	n	mpg	cyl	disp	hp	wt
1	5	18.50	6.80	223.40	198.80	3.00
2	8	28.08	4.00	93.61	81.88	2.04
3	7	20.74	5.14	175.11	102.14	3.19
4	12	15.05	8.00	357.62	194.17	4.10

Cluster	drat	qsec	vs	am	gear	carb
1	3.84	15.62	0.00	1	4.60	5.20
2	4.18	18.45	0.88	1	4.25	1.50
3	3.57	19.97	1.00	0	3.57	2.14
4	3.12	17.14	0.00	0	3.00	3.08

Here, `mpg` is miles per gallon, `disp` is displacement in cubic inches, `hp` is horsepower, `wt` is weight in 1,000 pounds, and `qsec` is quarter-mile time in seconds. The mean of `am` is the proportion with manual transmission; the mean of `vs` is the proportion with a straight engine. The other columns are cylinders (`cyl`), rear axle ratio (`drat`), forward gears (`gear`), and carburetors (`carb`).

Group characteristics and why standardization matters

Cluster 1: These five cars all have manual transmission. They have relatively high average horsepower (**198.80 hp**) and the shortest average quarter-mile time (**15.62 seconds**). The group includes the Mazda RX4 models and several higher-performance cars, so the cars are not identical, but they share some performance and transmission characteristics.

Cluster 2: These eight cars are lighter, with average weight **2.04 thousand pounds**, and all have four cylinders and manual transmission. Their average fuel economy is the highest at **28.08 mpg**, while average horsepower is the lowest at **81.88 hp**.

Cluster 3: These seven cars all have automatic transmission and straight engines. Their average fuel economy is **20.74 mpg** and average horsepower is **102.14 hp**. Their average quarter-mile time is the longest, at **19.97 seconds**.

Cluster 4: These 12 cars all have eight cylinders, automatic transmission, and three forward gears. They are the heaviest group on average (**4.10 thousand pounds**) and have the lowest average fuel economy (**15.05 mpg**). Their average horsepower is **194.17 hp**.

These differences support the four-group choice because the clusters separate lighter, more fuel-efficient cars from heavier eight-cylinder cars, while also showing differences in transmission and performance.

Standardization matters because the variables use different units and have very different numerical ranges. Without it, a variable such as displacement or horsepower could dominate Euclidean distance simply because its numbers vary more than variables such as `am` or `vs`. Standardizing puts each variable on a one-standard-deviation scale. This gives them comparable scale, although correlated variables can still carry overlapping information.